规培

Zynq 开发基础与 *Shell* 移植



Version: Beta

Author: OpenColinkLab

Date: September 2026

目录

1 概念扫盲 **2**

1.1 各个模块 . . . 2

1.1.1 PS . . . 2

1.1.2 PL . . . 2

1.1.3 AXI . . . 2

1.1.4 AXI-Lite . . . 2

1.1.5 Shell . . . 2

1.1.6 Block Design . . . 2

1.2 PS-PL 架构与数据流 . . . 3

2 硬件问题 **3**

2.1 电源 . . . 4

2.2 串口 . . . 4

2.3 下载器 . . . 4

3 注意事项 **4**

3.1 大致流程 . . . 4

3.2 手搓代码 . . . 5

4 移植 shell **5**

4.1 命令名 . . . 5

4.2 读函数 . . . 6

4.3 写函数 . . . 6

4.4 main 函数设置 . . . 6

5 硬件平台配置 **6**

5.1 PS-PL 架构 . . . 6

5.2 AXI 总线 . . . 7

5.3 DDR . . . 8

6 参考课程 **9**

1 概念扫盲

1.1 各个模块

1.1.1 PS

Processing System

处理系统, 即 Zynq 芯片中的 ARM 处理器部分。包含双核 Cortex-A9、内存控制器、外设接口（UART、SPI、I2C、以太网等）。PS 是" 软件侧", 运行 C 语言程序。

1.1.2 PL

Programmable Logic

可编程逻辑, 即 Zynq 芯片中的 FPGA 部分。基于 Artix-7 架构, 可以用 Verilog/VHDL 编程实现任意数字电路。PL 是" 硬件侧", 实现自定义逻辑。

1.1.3 AXI

Advanced eXtensible Interface

ARM 公司提出的片上总线协议, 是 AMBA 3.0/4.0 标准的一部分。分为 AXI-Full（高性能数据传输）、AXI-Lite（轻量寄存器访问）、AXI-Stream（流式数据）三种类型。Zynq 中 PS 与 PL 的主要通信通道都基于 AXI。

1.1.4 AXI-Lite

Advanced eXtensible Interface Lite

AXI 总线的轻量子集, 是 PS 与 PL 之间最常用的通信接口之一。采用寄存器映射方式,PS 端通过读写 PL 端的寄存器来控制硬件逻辑。适合低速、控制类应用, 如 LED 控制、寄存器配置等。

详细介绍见文档《ZYNQ-以规培项目为例》。

1.1.5 Shell

一种通过串口与用户交互的命令行界面。用户输入命令字符串, 系统解析并执行对应操作, 再通过串口返回结果。类似 Windows 的 CMD 或 Linux 的 Terminal, 但运行在嵌入式系统上。

1.1.6 Block Design

Vivado 中的图形化 IP 集成方式。通过拖拽和连线的方式, 将 Zynq PS、AXI 接口、自定义 IP 等模块连接起来, 构建完整的硬件系统。是 Zynq 开发中连接 PS 和 PL 的主要方式。

1.2 PS-PL 架构与数据流

本任务的核心是 PS 通过 AXI-Lite 总线控制 PL 端的 LED。理解数据如何从用户输入最终流向 LED 是关键。

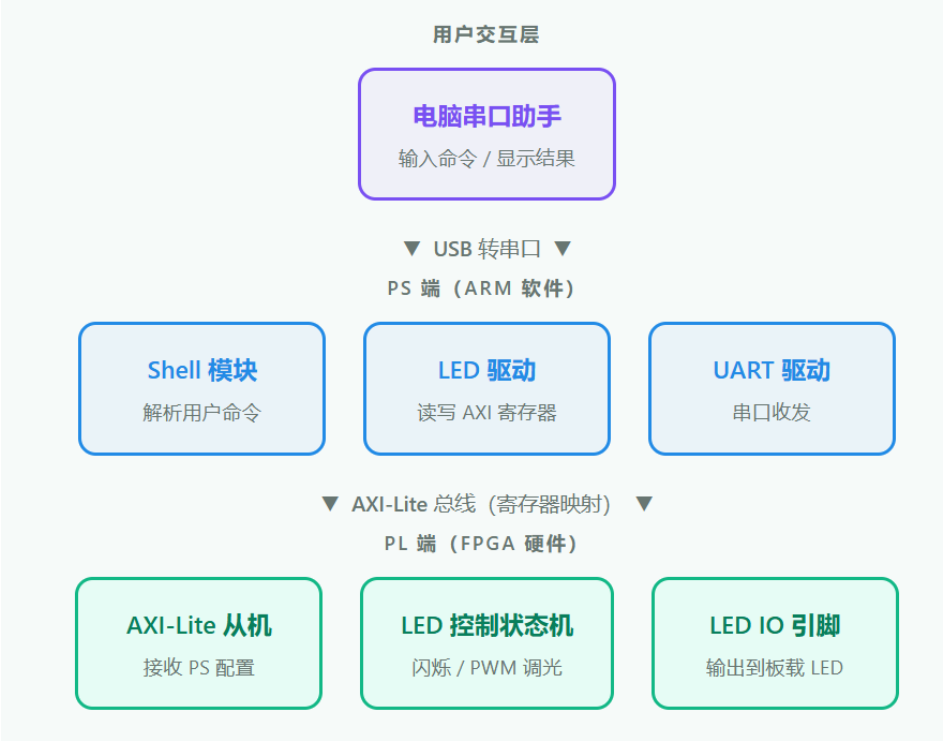


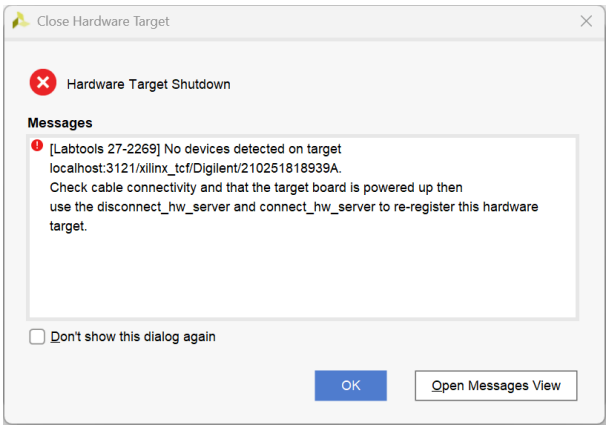
图 1: 整体架构

整个数据流可以概括为：用户在电脑上通过串口助手输入命令 → PS 端的 Shell 解析命令 → LED 驱动通过 AXI-Lite 总线写入 PL 端的控制寄存器 → PL 端的状态机根据寄存器值改变 LED 行为 → LED 引脚输出相应的电平或 PWM 信号 → LED 呈现出闪烁或调光效果。

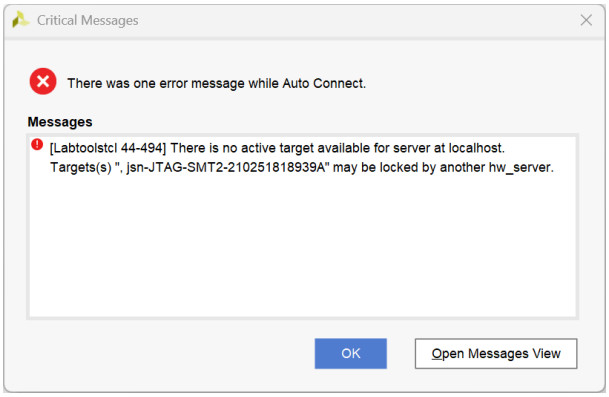
这部分详细可看《ZYNQ 硬件-以规培为例》。

2 硬件问题

使用板子时要解决三个问题, 分别关于电源、串口、下载器。这三个部分会影响 FPGA 的连接, 如有不慎会出现以下报错:



(a) 连接错误



(b) JTAG 未连接

图 2: 位置确定

2.1 电源

板子在 J4 接口处有专门的电源供电, 需要买电源适配器。使用 6pin 接口, 注意板子背面 D24 需要导通, 需要焊接。

另附：在尝试过程使用 20pin 的引脚进行供电, 那么此时需要 DCDC 进行电源供电。为了满足其供电需求,12V 和 400mA 以上,DCDC 模块的电源适配器选择应是 12V/2A, 保证输出的电压可以供板子。(不建议使用后者, 使用前者)

2.2 串口

在板子买回来后, 串口排针 J7 是空的, 需要进行焊接。在调试过程中需要用到串口调试，检查输出；最终结果时需要将串口连接电脑在 SDK Terminal 中输入命令控制。

2.3 下载器

不同于 typeC 接口, 该板子需要通过 JTAG 口进行烧录代码。

先在板子 J8 的接口将 14pin 的排针焊接完, 买下载器, 将排线和接口对齐接上。注意下载器的排线两头间距要适配焊的排针间距 (建议都用 2.54mm), 下载器注意 GND 和 Vref 不要接反, 会烧板。

初次上电后有三个灯亮表示正常：电源灯 LED4 和芯片灯 LED1 都会亮，同时板子的 PWM 灯也会亮，绿灯亮，红灯等一段时间闪烁。

3 注意事项

3.1 大致流程

在进行搭建过程时若时间充裕, 推荐如下流程：

1. 先在 vivado 平台搭建硬件，此时只加入 PS 来进行处理。这里简单设置串口，结合原理图，MIO 口是 24-25，这里配错也会导致烧录失败。在 SDK（这里 vivado 选择 18.3）里选择简单的 Hello World 代码用于串口测试，在 SDK Terminal 里查看到输出的结果即可。

2. 尝试点灯，这里对灯做一个选择，有 VGA 转接板可以选择 J18 和 H15，没有转接板就选择板子上的红灯 W13。

3. 若要进行 shell 移植，选择开源代码或实验室代码均可。先从简单命令了解大致流程后上升工程使用。(这部分检查类似于串口调试，发送和返回情况的日志)

4. 结合要求写代码，可以问 AI，但是不能代替工作，推荐古法编程。

3.2 手搓代码

关于整个流程中需要搞明白：

1. shell 的工作流程：其中要用什么函数？读写操作的格式是什么？执行操作是哪个函数？命令设定是什么（保证命令被识别)?

2. FPGA 部分：有没有给 AXI 总线接口设置地址?（这里以灯控代码写在 PL 端为例）灯控.v 文件是否需要放进.bd 图里一起打包？要实现 AXI 总线通信都需要添加哪些模块？模块怎么配置？模块和模块之间怎么接线?

3. SDK 部分：给 shellCommand 内存布局了吗（即在.ld 文件里是否添加起始地址索引和终点地址索引)？移植后有没有需要修改的参数？读函数中怎么判断 FIFO 里有没有数据？写函数里怎么判断 FIFO 里有没有空位，想要完成这两项功能要看什么文件？对于读写寄存器的位置是选择写死还是用结构体表示?

   ...

4 移植 shell

以移植 GitHub 开源 shell 为例，移植版本 3.2.4.

用户输入命令 -> 通过 read/write 函数写数据 -> 分别在 task 函数里解析和处理命令和数据 -> 观察结果

4.1 命令名

这就意味着输入的命令需要名字，文件 shell.h 中：

```
#define SHELL_EXPORT_CMD(_attr, _name, _func, _desc)
```

这个宏定义在给自己建立的命令进行设置并纳入 shellCommandList 中。这四个参数的意思分别是：

表 1: 参数解释

参数	名字	类型	作用	举例
参数 1	_attr	属性（数字）	命令的权限、类型等属性	SHELL_CMD_PERMISSION() | SHELL_CMD_TYPE(SHELL_TYPE_CMD_MAIN)
参数 2	_name	标识符	命令名（你在串口里输入的名字）	led
参数 3	_func	函数指针	对应的 C 函数名（执行命令时调用这个函数）	led_cmd
参数 4	_desc	字符串	命令描述（help 命令时显示的帮助文字）	“LED control command”

4.2 读函数

在 shell.h 文件里提到了读写函数设置规则:

```
signed short (*read)(char *, unsigned short);
signed short (*write)(char *, unsigned short);
```

在此引入两个参量，即数据指针和字符长度，通过 xuartps.h 文件中的函数去进行数据读取，这里需要设置读取地址，推荐使用结构体，这样可以在换平台是由运行 bsp 自动获取，防止出错。每次读取完要返回 1，没有读取到返回 0。

4.3 写函数

设置原理同上。

4.4 main 函数设置

和 STM32 中初始化很像，对 shell 进行初始化，调用函数；对串口进行初始化，这里函数需要获取设备地址，可以用带有的 Lookup 函数进行查找。while 里需要包括 shell 处理函数。

5 硬件平台配置

本文仅针对 ZYNQ 的硬件部分做整体陈述，关于在本次实践中学到的新部分加以深入探究。

5.1 PS-PL 架构

ZYNQ 集成两个功能，同时拥有 FPGA 和 ARM，这两者通过 AXI 协议进行交流。和单纯使用 FPGA 协同 STM32 相比，这样的通信更稳定，也可以更好地使用一块板子用于数据处理和逻辑控制进

行架构分区，即 PS 处理复杂的逻辑控制部分，PL 做数据运算。AXI 总线可以使数据在这两个部分中流动，使用 PS 产生经过 PLL 分频的时钟，在工程中本文配置管脚为 $FCLK_CLK0$。

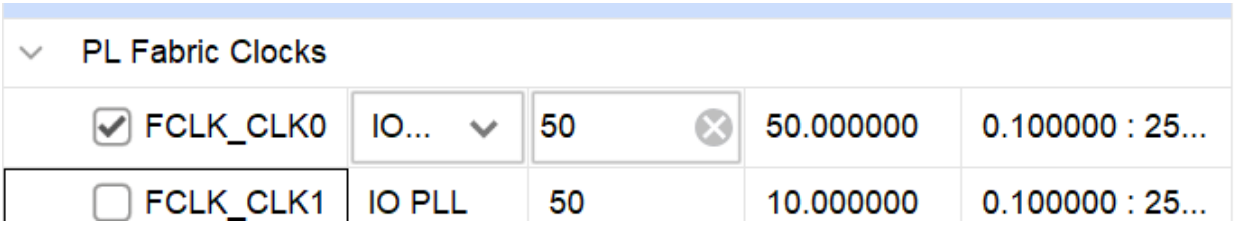


图 3: 配置情况

5.2 AXI 总线

AXI 作为数据流通通路，完成数据传输、控制读写的功能。在 vivado 工程中有专门的 IP 核叫 axi_interconnect 和它配套的翻译 PS 成 PL 语言的 IP 核是 AXI GPIO, 两者搭配进行翻译。在实验室提供的代码中给了 AXI-Lite，属于一个轻量的简化子集，在使用时注意事项请咨询大佬，本文不做举例。

除此之外，AXI 分类不止 AXI-Lite，还有面对大量数据例如运输 DMA 时采用的 AXI-Full 和纯数据流传输的 AXI-Stream。

以配置 IP 核为例，接线如下：

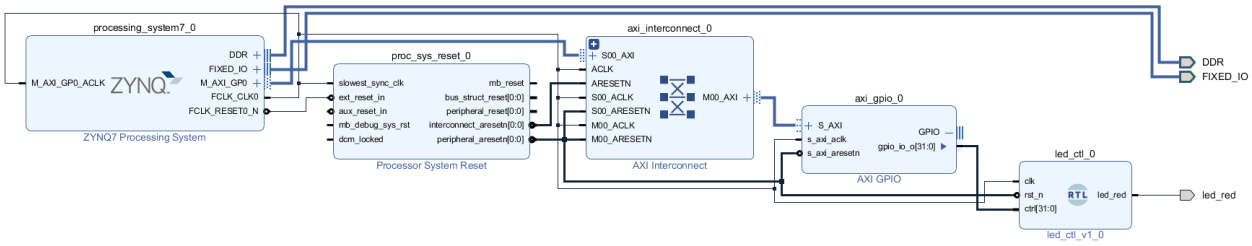


图 4: 接线

在 PS 的 IP 核中，要设置外设（串口、SPI、I2C 等），在 PeripheralI/O Pins 里进行配置，这里调用串口我们采用 24-25IO 口，在 MIO Configuration 进行配置；PS-PL 通信需要在 PS-PL Configuration 中进行配置，本工程中基本默认即可；Clock Configuration 时钟频率在这里进行调整，有针对 CPU 的、DDR 的，本例不做调整；DDR Configuration 是对 DDR 的参数进行调整，注意我们这里采用 16bit 位宽的。

在 AXI-Lite 设计时需要注意握手协议、地址对齐等等，详细设计请参考这个：

AMBA 总线-AXI 协议

这里做简单总结：

1. AXI 协议将控制和地址分离，通过 AXI 找寻地址，例如功能地址，再通过 AXI-Lite 进行数据传输。在 Master 和 Slave 端分别有五个信号：

表 2: AXI 协议五个独立事务通道说明

名称	方向	作用
AW（写地址）	主 → 从	写在哪
W（写数据）	主 → 从	写什么
B（写响应）	从 → 主	写好了吗
AR（读地址）	主 → 从	读哪
R（读数据）	从 → 主	读什么

整个握手过程分为三个情况：VALID 等 READY，READY 等 VALID，两者同时。在读时需要无论 AXI3 还是 AXI4 都是先有地址，后有数据；在写时，AXI3 和 AXI4 不同的点在于前者写数据在前，响应在后，后者在数据和地址在前，响应在后。

2. AXI 可以处理突发事件，用 AxSIZE 表示突发大小，AxBURST 定义 3 种突发类型。

3. AXI-Lite 是 AXI 的简化版，用于寄存器的配置，不支持突发事件，简化后的信号为：

Global	Write address channel	Write data channel	Write response channel	Read address channel	Read data channel
ACLK	AWVALID	WVALID	BVALID	ARVALID	RVALID
ARESETn	AWREADY	WREADY	BREADY	ARREADY	RREADY
–	AWADDR	WDATA	BRESP	ARADDR	RDATA
–	AWPROT	WSTRB	–	ARPROT	RRESP

公众号：处理器+AI知识星球

图 5: AXI4-Lite 信号

而 AXI-Stream 是最简化的，不问地址，只传数据。一般使用 AXI-Lite 搭配这个使用，前者负责控制，后者负责传输，例如前者配置 FFT 的参数，后者将将数据传入 FFT 再传入后级。(这里额外插一句，前段时间在了解高速传输的串行总线时偶然知道 PCIe，这里也在 FPGA 中广泛应用，和上述总线协议不同的是，这里是跨设备的，例如将 PC 端数据传输到 FPGA 中的 AXI-Lite 或者 AXI-Stream 会用到)

5.3 DDR

全称比较长，这三个字母是 Double Data Rate 的缩写，双倍的原因是在一个周期内上升沿和下降都进行数据传输，这块板子是 DDR3，即 8bit 位宽，目前已经发展到了 DDR4、DDR5，位宽在不断拓宽。DDR 作为 RAM 最主流的一种，其位宽不断扩大，存储量也在增加，以我们使用的这块芯片为例：

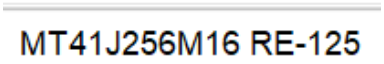


图 6: DDR3 芯片

每个 Bank 里有 256 个地址，一个有 16 个 Bank，总共容量达到 256×16 $4Gbit$，即 512MB。

这里引用前辈的话：DDR 内存的进化方向逐渐清晰：更高的数据速率、降低工作电压、并通过内部架构变化避免 DRAM 核心频率无限延长。

6 参考课程

详细 verilog 语法和 Vivado 使用可参考《PG002 规培任务》中的指导课程。这里推荐几个关于 ZYNQ 学习（EBAZ4205 建设）的指导（包含调用串口、PS-PL 工程零基础、也有引脚图等等）：

ZYNQ 矿板开发教程-低成本开发板学习 ZYNQ

第 14-19 讲 _ZYNQ 输入输出实验 _LED 闪烁、按键控制、蜂鸣器、呼吸灯

从零开始的 ZYNQ 学习（基于矿卡 EBAZ4205）（一）

EBZA4205 001 学习点灯

C 语言指针、结构体推荐：

9.1.2 指针

第五十九节指针变量的赋值方式

第八十二节之自定义数据类型之结构体概念

树莓派推荐指导如下（包含系统安装、远程控制、基础工程）：

零基础玩转树莓派

【王铭东】Python+ 树莓派 4+Linux 快速上手

这里建议使用 SSH 登录时在 VScode 上拓展下载 SSH 修改里面的 config 文件，这样代码可以在这里写（虚拟机下载太慢了，尽量放本机）。